

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ФАКУЛЬТЕТ ПРИКЛАДНОЙ МАТЕМАТИКИ И ИНФОРМАТИКИ

ОТЧЕТ
по лабораторной работе № 1
Цветовые модели

Хургиной Софии
Владимировны
студентки 3 курса, 2
группы
специальность
«Информатика»

Преподаватель:
Каркоцкий А. Г.

Минск, 2026

1. Цель работы

Целью лабораторной работы является разработка графического приложения для работы с цветовыми моделями **RGB**, **CMYK** и **HSV**.

В процессе выполнения работы необходимо реализовать:

- преобразование цвета между цветовыми моделями RGB, CMYK и HSV;
- отображение компонентов цветовых моделей в графическом интерфейсе;
- изменение цвета с помощью ползунков и текстовых полей;
- отображение текущего цвета в области предварительного просмотра;
- открытие палитры;
- синхронизацию значений цветовых моделей при изменении цвета.

2. Постановка задачи

Необходимо разработать приложение для конвертации цветов между тремя цветовыми моделями:

- **RGB** — Аддитивная модель; красный, зеленый и синий компоненты;
- **CMYK** — Субтрактивная модель; голубой, пурпурный, желтый и черный компоненты;
- **HSV** — Перцепционная модель; оттенок, насыщенность и значение.

Пользователь должен иметь возможность изменять значения компонентов цвета с помощью элементов графического интерфейса. При изменении одной модели остальные модели должны автоматически пересчитываться.

3. Используемые средства

Для выполнения лабораторной работы использовались:

- язык программирования **Java**;
- библиотека графического интерфейса **Java Swing**;
- класс **JColorChooser** для выбора цвета;
- классы **JFrame**, **JPanel**, **JSlider**, **TextField**, **Button** и другие компоненты **Swing**;

- объектная модель приложения с разделением на модель данных и графическое представление.

Проект состоит из следующих файлов:

Файл	Назначение
ColorConverter.java	Реализация математических преобразований цветовых моделей
ColorModel.java	Хранение текущего цвета и синхронизация моделей
ColorView.java	Основные элементы графического интерфейса
ModelPanel.java	Панели с ползунками и полями для компонентов цвета
MainFrame.java	Главное окно и обработка взаимодействия пользователя
ColorConverterApp.java	Точка входа в приложение

4. Ход выполнения работы

4.1. Реализация преобразования RGB → HSV

Для преобразования RGB в HSV сначала значения компонентов RGB нормализуются в диапазон от 0 до 1.

Определяются максимальное и минимальное значения:

```
double max = Math.max(r, Math.max(g, b));
double min = Math.min(r, Math.min(g, b));
double delta = max - min;
```

Значение яркости V определяется как максимальное значение:

```
v = max;
```

Насыщенность вычисляется следующим образом:

```
if (max == 0) {
    s = 0;
} else {
    s = delta / max;
}
```

Оттенок определяется в зависимости от того, какой из RGB-компонентов является максимальным. После вычисления оттенка отрицательное значение корректируется добавлением 360 градусов.

4.2. Реализация преобразования HSV → RGB

Для обратного преобразования используется стандартное разбиение цветового круга HSV на шесть секторов:

```
if (h < 60) {  
    ...  
} else if (h < 120) {  
    ...  
} else if (h < 180) {  
    ...  
} else if (h < 240) {  
    ...  
} else if (h < 300) {  
    ...  
} else {  
    ...  
}
```

После вычисления компонентов RGB они переводятся из диапазона 0..1 в диапазон 0..255.

4.3. Реализация преобразований RGB и CMYK

Для преобразования RGB в CMYK сначала RGB-компоненты переводятся в диапазон 0..1.

Черный компонент вычисляется как:

```
k = 1 - Math.max(r, Math.max(g, b));
```

После этого вычисляются значения компонентов C, M и Y.

Для обратного преобразования CMYK → RGB используются формулы:

```
r = 255 * (1 - c) * (1 - k);  
g = 255 * (1 - m) * (1 - k);  
b = 255 * (1 - y) * (1 - k);
```

4.4. Создание модели данных

Класс `ColorModel` отвечает за хранение текущих значений RGB, CMYK и HSV.

При изменении RGB выполняется пересчет CMYK и HSV:

```
this.cmyk = ColorConverter.rgbToCmyk(this.rgb);
```

```
double oldHue = this.hsv[0];
double[] newHsv = ColorConverter.rgbToHsv(this.rgb);

if (newHsv[0] == 0 && Math.abs(oldHue - 360.0) < 0.001)
{
    newHsv[0] = 360.0;
}

this.hsv = newHsv;
notifyListeners();
```

Здесь реализована дополнительная обработка значения 360° .

Математически оттенки 0° и 360° соответствуют одному и тому же цвету. Однако для интерфейса лабораторной работы необходимо сохранить введенное пользователем значение 360° . Поэтому, если новый оттенок рассчитывается как 0° , но предыдущий оттенок был 360° , сохраняется значение 360° .

4.5. Создание графического интерфейса

Для каждой цветовой модели создается отдельная панель с ползунками и текстовыми полями.

Для HSV используются следующие диапазоны:

Компонент	Диапазон
H	0–360
S	0–100
V	0–100

Для RGB диапазон каждого компонента составляет 0–255.

Для CMYK используются значения компонентов в диапазоне 0–100.

При изменении значения пользовательского элемента вызывается соответствующий метод модели, после чего остальные элементы интерфейса обновляются.

4.6. Работа со стандартной палитрой JColorChooser

Для выбора цвета используется стандартный компонент Java:

```
JColorChooser chooser = new  
JColorChooser(model.getAwtColor());
```

Для открытия окна выбора цвета создается диалог:

```
JDialog dialog = JColorChooser.createDialog(  
    MainFrame.this,  
    "Выберите цвет",  
    true,  
    chooser,  
    null,  
    null  
);
```

После выбора цвета его RGB-компоненты передаются в модель:

```
model.setRgb(new double[]{  
    selectedColor.getRed(),  
    selectedColor.getGreen(),  
    selectedColor.getBlue()  
});
```

Таким образом, выбранный в палитре цвет становится текущим цветом приложения.

4.7. Обработка значения $HSV = 360^\circ$

В ходе выполнения лабораторной работы была обнаружена особенность преобразования HSV.

Цветовой круг является циклическим, поэтому:

$H = 0^\circ$

и

$H = 360^\circ$

представляют один и тот же оттенок.

При стандартном преобразовании $RGB \rightarrow HSV$ красный цвет возвращается с оттенком 0° . Однако в рамках лабораторной работы требовалось сохранить введенное пользователем значение 360° .

Для этого в `ColorModel.java` было изменено присваивание результата преобразования:

```
this.hsv = newHsv;
```

ВМЕСТО ПОВТОРНОГО ВЫЗОВА:

```
this.hsv = ColorConverter.rgbToHsv(this.rgb);
```

Благодаря этому сохраненное значение 360° не перезаписывается значением 0° .

Также в `MainFrame.java` была добавлена настройка стандартного `JColorChooser`, позволяющая перед открытием палитры установить текущее значение оттенка HSV.

5. Исходный код

`ColorConverter.java`

```
1. public class ColorConverter {
2.
3.     public static double[] rgbToHsv(double[] rgb) {
4.         double rN = rgb[0] / 255.0;
5.         double gN = rgb[1] / 255.0;
6.         double bN = rgb[2] / 255.0;
7.
8.         double max = Math.max(rN, Math.max(gN, bN));
9.         double min = Math.min(rN, Math.min(gN, bN));
10.        double delta = max - min;
11.
12.        double h = 0;
13.        if (delta != 0) {
14.            if (max == rN) {
15.                h = 60 * (((gN - bN) / delta) % 6);
16.            } else if (max == gN) {
17.                h = 60 * (((bN - rN) / delta) + 2);
18.            } else {
19.                h = 60 * (((rN - gN) / delta) + 4);
20.            }
21.        }
22.        if (h < 0) h += 360;
23.
24.        double s = (max == 0) ? 0 : (delta / max) * 100;
25.        double v = max * 100;
26.
27.        return new double[]{h, s, v};
28.    }
29.
30.    public static double[] hsvToRgb(double[] hsv) {
31.        double sN = hsv[1] / 100.0;
32.        double vN = hsv[2] / 100.0;
33.
34.        double c = vN * sN;
35.        double x = c * (1 - Math.abs((((hsv[0] / 60.0) % 2) - 1)));
36.        double m = vN - c;
37.
38.        double r = 0, g = 0, b = 0;
39.        if (hsv[0] >= 0 && hsv[0] < 60) { r = c; g = x; b = 0; }
```

```

40.         else if (hsv[0] >= 60 && hsv[0] < 120) { r = x; g = c; b =
    0; }
41.         else if (hsv[0] >= 120 && hsv[0] < 180) { r = 0; g = c; b
    = x; }
42.         else if (hsv[0] >= 180 && hsv[0] < 240) { r = 0; g = x; b
    = c; }
43.         else if (hsv[0] >= 240 && hsv[0] < 300) { r = x; g = 0; b
    = c; }
44.         else if (hsv[0] >= 300 && hsv[0] <= 360) { r = c; g = 0; b
    = x; }
45.
46.         return new double[]{(r + m) * 255, (g + m) * 255, (b + m)
    * 255};
47.     }
48.
49.     public static double[] rgbToCmyk(double[] rgb) {
50.         double rN = rgb[0] / 255.0;
51.         double gN = rgb[1] / 255.0;
52.         double bN = rgb[2] / 255.0;
53.
54.         double k = 1.0 - Math.max(rN, Math.max(gN, bN));
55.         double c = (k == 1.0) ? 0 : (1.0 - rN - k) / (1.0 - k);
56.         double m = (k == 1.0) ? 0 : (1.0 - gN - k) / (1.0 - k);
57.         double y = (k == 1.0) ? 0 : (1.0 - bN - k) / (1.0 - k);
58.
59.         return new double[]{c * 100, m * 100, y * 100, k * 100};
60.     }
61.
62.     public static double[] cmykToRgb(double[] cmyk) {
63.         double cN = cmyk[0] / 100.0;
64.         double mN = cmyk[1] / 100.0;
65.         double yN = cmyk[2] / 100.0;
66.         double kN = cmyk[3] / 100.0;
67.
68.         double r = 255 * (1 - cN) * (1 - kN);
69.
70.         double g = 255 * (1 - mN) * (1 - kN);
71.         double b = 255 * (1 - yN) * (1 - kN);
72.
73.         return new double[]{r, g, b};
    }
}

```

ColorModel.java:

```

1. public class ColorModel {
2.
3.     public interface ColorChangeListener {
4.         void onColorChanged(ColorModel model);
5.     }
6.
7.     private double[] rgb = new double[]{255, 0, 0};
8.     private double[] cmyk = new double[]{0, 100, 100, 0};
9.     private double[] hsv = new double[]{0, 100, 100};
10.
11.     private final List<ColorChangeListener> listeners = new
    ArrayList<>();
12.
13.     public void addListener(ColorChangeListener listener) {
14.         listeners.add(listener);
15.     }
}

```



```

16.
17.     private void notifyListeners() {
18.         for (ChangeListener listener : listeners) {
19.             listener.onColorChanged(this);
20.         }
21.     }
22.
23.     public void setRgb(double[] rgb) {
24.         this.rgb[0] = Math.max(0, Math.min(255, rgb[0]));
25.         this.rgb[1] = Math.max(0, Math.min(255, rgb[1]));
26.         this.rgb[2] = Math.max(0, Math.min(255, rgb[2]));
27.
28.         this.cmyk = ColorConverter.rgbToCmyk(this.rgb);
29.         double oldHue = this.hsv[0];
30.         double[] newHsv = ColorConverter.rgbToHsv(this.rgb);
31.
32.         if (newHsv[0] == 0 && Math.abs(oldHue - 360.0) < 0.001) {
33.             newHsv[0] = 360.0;
34.         }
35.
36.         this.hsv = newHsv;
37.         notifyListeners();
38.     }
39.
40.     public void setCmyk(double[] cmyk) {
41.         this.cmyk = Arrays.copyOf(cmyk, cmyk.length);
42.         this.rgb = ColorConverter.cmykToRgb(this.cmyk);
43.         this.hsv = ColorConverter.rgbToHsv(this.rgb);
44.         notifyListeners();
45.     }
46.
47.     public void setHsv(double[] hsv) {
48.         this.hsv = Arrays.copyOf(hsv, hsv.length);
49.         this.rgb = ColorConverter.hsvToRgb(this.hsv);
50.         this.cmyk = ColorConverter.rgbToCmyk(this.rgb);
51.         notifyListeners();
52.     }
53.
54.     public double[] getRgb() { return Arrays.copyOf(rgb,
rgb.length); }
55.     public double[] getCmyk() { return Arrays.copyOf(cmyk,
cmyk.length); }
56.     public double[] getHsv() { return Arrays.copyOf(hsv,
hsv.length); }
57.
58.     public Color getAwtColor() {
59.         return new Color((int) Math.round(rgb[0]), (int)
Math.round(rgb[1]), (int) Math.round(rgb[2]));
60.     }
61. }

```

ColorView.java:

```

1. public class ColorView extends JPanel {
2.
3.     private final JPanel previewBox;
4.     private final JButton chooseColorBtn;
5.     private final JTextField colorHexField;
6.
7.     public ColorView() {
8.         setLayout(new FlowLayout(FlowLayout.CENTER, 15, 10));
9.
10.         previewBox = new JPanel();

```

```

11.         previewBox.setPreferredSize(new Dimension(120, 60));
12.         previewBox.setBorder(BorderFactory.createLineBorder(Color.
    BLACK, 5));
13.
14.         chooseColorBtn = new JButton("Палитра");
15.
16.         colorHexField = new JTextField(10);
17.         colorHexField.setEditable(false);
18.         colorHexField.setHorizontalAlignment(JTextField.CENTER);
19.
20.         add(colorHexField);
21.         add(previewBox);
22.         add(chooseColorBtn);
23.     }
24.
25.     public JButton getChooseColorBtn() {
26.         return chooseColorBtn;
27.     }
28.
29.     public void updateColor(Color color) {
30.         previewBox.setBackground(color);
31.         String hex = String.format("#%02X%02X%02X",
    color.getRed(), color.getGreen(), color.getBlue());
32.         colorHexField.setText(hex);
33.         previewBox.repaint();
34.     }
35. }

```

ModelPanel.java:

```

1. public class ModelPanel extends JPanel {
2.
3.     private final JSlider[] sliders;
4.     private final JTextField[] fields;
5.     private boolean isUpdating = false;
6.     private Consumer<double[]> onValuesChanged;
7.
8.     public ModelPanel(String title, String[] labels, double[] minVals,
    double[] maxVals) {
9.         setLayout(new GridBagLayout());
10.        setBorder(BorderFactory.createTitledBorder(
11.            BorderFactory.createEtchedBorder(), title,
    TitledBorder.LEFT, TitledBorder.TOP,
12.            new Font("SansSerif", Font.BOLD, 12)
13.        ));
14.
15.        int n = labels.length;
16.        sliders = new JSlider[n];
17.        fields = new JTextField[n];
18.
19.        GridBagConstraints gbc = new GridBagConstraints();
20.        gbc.insets = new Insets(5, 5, 5, 5);
21.        gbc.fill = GridBagConstraints.HORIZONTAL;
22.
23.        for (int i = 0; i < n; i++) {
24.            gbc.gridx = 0; gbc.gridy = i; gbc.weightx = 0;
25.            add(new JLabel(labels[i]), gbc);
26.
27.            sliders[i] = new JSlider((int) minVals[i], (int)
    maxVals[i], (int) minVals[i]);
28.            gbc.gridx = 1; gbc.weightx = 1.0;
29.            add(sliders[i], gbc);
30.

```

```

31.         fields[i] = new JTextField(5);
32.         fields[i].setHorizontalAlignment(JTextField.RIGHT);
33.         gbc.gridx = 2; gbc.weightx = 0;
34.         add(fields[i], gbc);
35.
36.         final int index = i;
37.
38.
39.         sliders[i].addChangeListener(new ChangeListener() {
40.             @Override
41.             public void stateChanged(ChangeEvent e) {
42.                 if (isUpdating) return;
43.                 fields[index].setText(String.format("%.1f",
(double) sliders[index].getValue()).replace(',', ' '));
44.                 triggerUpdate();
45.             }
46.         });
47.
48.         // Текстовое поле -> Обновление
49.         fields[i].addActionListener(new ActionListener() {
50.             @Override
51.             public void actionPerformed(ActionEvent e) {
52.                 if (isUpdating) return;
53.                 try {
54.                     double val =
Double.parseDouble(fields[index].getText().replace(',', ' '));
55.                     val = Math.max(minVals[index],
Math.min(maxVals[index], val));
56.                     sliders[index].setValue((int)
Math.round(val));
57.                     triggerUpdate();
58.                 } catch (NumberFormatException ex) {
59.                     fields[index].setText(String.format("%.1f"
, (double) sliders[index].getValue()).replace(',', ' '));
60.                 }
61.             }
62.         });
63.     }
64. }
65.
66. public void setOnValuesChanged(Consumer<double[]> callback) {
67.     this.onValuesChanged = callback;
68. }
69.
70. private void triggerUpdate() {
71.     if (onValuesChanged == null) return;
72.     double[] values = new double[sliders.length];
73.     for (int i = 0; i < sliders.length; i++) {
74.         values[i] = sliders[i].getValue();
75.     }
76.     onValuesChanged.accept(values);
77. }
78.
79. public void setValues(double[] values) {
80.     isUpdating = true;
81.     for (int i = 0; i < values.length; i++) {
82.         sliders[i].setValue((int) Math.round(values[i]));
83.         fields[i].setText(String.format("%.1f",
values[i]).replace(',', ' '));
84.     }
85.     isUpdating = false;
86. }
87. }

```

MainFrame.java:

```
1. public class MainFrame extends JFrame {
2.
3.     private final ColorModel model;
4.     private boolean isSelfUpdating = false;
5.
6.     private ColorView colorView;
7.     private ModelPanel rgbPanel;
8.     private ModelPanel cmykPanel;
9.     private ModelPanel hsvPanel;
10.
11.     public MainFrame(ColorModel model) {
12.         this.model = model;
13.
14.         setTitle("Color Converter App");
15.         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
16.         setLayout(new BorderLayout(10, 10));
17.
18.         initComponents();
19.         setupListeners();
20.
21.         model.setRgb(new double[]{255, 0, 0});
22.
23.         pack();
24.         setLocationRelativeTo(null);
25.         setResizable(false);
26.     }
27.
28.     private void initComponents() {
29.         colorView = new ColorView();
30.
31.         rgbPanel = new ModelPanel("RGB (0..255)",
32.             new String[]{"R:", "G:", "B:"},
33.             new double[]{0, 0, 0},
34.             new double[]{255, 255, 255});
35.
36.         cmykPanel = new ModelPanel("CMYK (0..100%)",
37.             new String[]{"C:", "M:", "Y:", "K:"},
38.             new double[]{0, 0, 0, 0},
39.             new double[]{100, 100, 100, 100});
40.
41.         hsvPanel = new ModelPanel("HSV (0..360°, 0..100%)",
42.             new String[]{"H:", "S:", "V:"},
43.             new double[]{0, 0, 0},
44.             new double[]{360, 100, 100});
45.
46.         JPanel modelsContainer = new JPanel(new GridLayout(1, 3,
47.             10, 0));
48.         modelsContainer.setBorder(BorderFactory.createEmptyBorder(
49.             0, 10, 10, 10));
50.         modelsContainer.add(rgbPanel);
51.         modelsContainer.add(cmykPanel);
52.         modelsContainer.add(hsvPanel);
53.
54.         add(colorView, BorderLayout.NORTH);
55.         add(modelsContainer, BorderLayout.CENTER);
56.     }
57.
58.     private void setupListeners() {
59.         model.addListener((m) -> {
60.             if (isSelfUpdating) return;
61.             isSelfUpdating = true;
```

```

60.
61.         colorView.updateColor(m.getAwtColor());
62.         rgbPanel.setValues(m.getRgb());
63.         cmykPanel.setValues(m.getCmyk());
64.         hsvPanel.setValues(m.getHsv());
65.
66.         isSelfUpdating = false;
67.     });
68.
69.     rgbPanel.setOnValuesChanged(vals -> {
70.         if (isSelfUpdating) return;
71.         model.setRgb(vals);
72.     });
73.
74.     cmykPanel.setOnValuesChanged(vals -> {
75.         if (isSelfUpdating) return;
76.         model.setCmyk(vals);
77.     });
78.
79.     hsvPanel.setOnValuesChanged(vals -> {
80.         if (isSelfUpdating) return;
81.         model.setHsv(vals);
82.     });
83.
84.     colorView.getChooseColorBtn().addActionListener(new
    ActionListener() {
85.         @Override
86.         public void actionPerformed(ActionEvent e) {
87.             openJavaColorChooser();
88.         }
89.     });
90. }
91.
92.     private void openJavaColorChooser() {
93.         final double[] currentHsv = model.getHsv();
94.
95.         final JColorChooser chooser = new
    JColorChooser(model.getAwtColor());
96.
97.         // Стандартная Java HSV-панель сначала получает H=0 для
    чистого красного.
98.         // Если в нашей модели был H=360, возвращаем 360 штатному
    HSV-слайдеру.
99.         setJavaChooserHue(chooser, currentHsv[0]);
100.
101.         ActionListener okListener = e -> {
102.             Color selectedColor = chooser.getColor();
103.             if (selectedColor != null) {
104.                 model.setRgb(new double[]{
105.                     selectedColor.getRed(),
106.                     selectedColor.getGreen(),
107.                     selectedColor.getBlue()
108.                 });
109.             }
110.         };
111.
112.         JDialog dialog = JColorChooser.createDialog(
113.             this,
114.             "Выберите цвет",
115.             true,
116.             chooser,
117.             okListener,
118.             null
119.         );

```

```

120.
121.         dialog.setVisible(true);
122.     }
123.
124.     private void setJavaChooserHue(JColorChooser chooser, double
hue) {
125.         if (Math.abs(hue - 360.0) > 0.001 && Math.abs(hue) >
0.001) {
126.             return;
127.         }
128.
129.         for (AbstractColorChooserPanel panel :
chooser.getChooserPanels()) {
130.             if (!"HSV".equalsIgnoreCase(panel.getDisplayName())) {
131.                 continue;
132.             }
133.
134.             List<JSlider> sliders = new ArrayList<>();
135.             collectSliders(panel, sliders);
136.
137.
138.             if (!sliders.isEmpty()
139.                 && sliders.get(0).getMinimum() == 0
140.                 && sliders.get(0).getMaximum() == 360) {
141.                 sliders.get(0).setValue(hue >= 359.999 ? 360 : 0);
142.             }
143.             return;
144.         }
145.     }
146.
147.     private void collectSliders(Component component, List<JSlider>
result) {
148.         if (component instanceof JSlider) {
149.             result.add((JSlider) component);
150.         }
151.
152.         if (component instanceof Container) {
153.             for (Component child : ((Container)
component).getComponents()) {
154.                 collectSliders(child, result);
155.             }
156.         }
157.     }
158. }

```

ColorConverterApp.java

```

1. public class ColorConverterApp {
2.     public static void main(String[] args) {
3.         try {
4.             UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClas
sName());
5.         } catch (Exception ignored) {}
6.
7.         SwingUtilities.invokeLater(() -> {
8.             ColorModel model = new ColorModel();
9.             MainFrame frame = new MainFrame(model);
10.            frame.setVisible(true);
11.        });
12.    }
13. }

```

6. Результат работы программы

В результате выполнения лабораторной работы было получено графическое Java-приложение, позволяющее:

1. Задавать цвет в модели RGB.
2. Задавать цвет в модели CMYK.
3. Задавать цвет в модели HSV.
4. Автоматически пересчитывать остальные цветовые модели.
5. Изменять компоненты цвета с помощью ползунков и текстовых полей.
6. Просматривать текущий цвет в области предварительного просмотра.
7. Выбирать цвет с помощью стандартной палитры JColorChooser.
8. Сохранять значение оттенка HSV 360° , не заменяя его автоматически на 0° .

7. Вывод

В ходе лабораторной работы было разработано графическое приложение на языке Java с использованием библиотеки Swing для конвертации цветов между моделями **RGB**, **CMYK** и **HSV**.

Были реализованы алгоритмы преобразования между цветовыми моделями, создан графический интерфейс с ползунками и текстовыми полями, а также реализована синхронизация данных между различными представлениями цвета.

Дополнительно была реализована работа со стандартным компонентом JColorChooser. Особенность циклического диапазона оттенка HSV была учтена таким образом, чтобы значение **360° сохранялось и отображалось как 360°** , несмотря на математическую эквивалентность значений 0° и 360° .

В результате поставленная задача была выполнена, а приложение позволяет удобно изменять и конвертировать цвет между различными цветовыми моделями.